Chapter 6

TypeScript

In programming, the *type* of a piece of variable data indicates the set of values that are valid for that data. For example, if I say that a variable is an integer, that means I expect the variable to have a value like `1`, `-3`, or `357` and not a value like `banana`. If the variable is set to a value that is not of the expected type, we think that something bad will happen. In some languages, the attempt to set data to the wrong value leads to a compiler error; in other languages it leads to incorrect or unspecified behavior at run time.

All high-level programing languages use some concept of type to determine behavior. Which is to say that all of these languages determine behavior by using not just the value of a variable in the system but also information about what kind of value it is.

TypeScript is a superset of JavaScript that optionally allows you to add annotations to the code to specify type information.[49] TypeScript requires a compilation step that enforces type consistency and converts valid TypeScript to JavaScript suitable for browsers. The type system that TypeScript uses makes inferences about types based on the code, even if you do not explicitly provide type information. The goal of using TypeScript is to reduce code errors, first by preventing type mismatches, and as you become more advanced, by making invalid states into compiler-time errors so they are impossible to achieve at run time. TypeScript only enforces type consistency at compile time; it provides no protection against things that might happen at run time.

In this chapter, we'll cover the basics of TypeScript's syntax and semantics and